

Lending Decision System

for MSMEs

A product thinking + engineering exercise to simulate how digital lenders process applications and generate credit decisions. You will build a complete, deployable system — backend logic, API layer, and frontend interface — within a single working day.

DURATION 1 working day	SUBMISSION GitHub + deployed preview + write-up PDF	TECH STACK React · Node/Express · PostgreSQL + MongoDB	CONTACT Submit via email to your recruiter
----------------------------------	---	--	--

00 OBJECTIVE

Build a lightweight, end-to-end lending decision system that accepts MSME business profiles and loan inputs, runs them through a credit decision engine, and surfaces a structured decision with reasoning. The emphasis is on clean architecture, real-world constraints, and thoughtful product judgment — not feature volume.

01 FUNCTIONAL REQUIREMENTS

BE Backend API Layer

Required

Business & user profile endpoint

- Business owner name and PAN (mock format acceptable)
- Business type (e.g. retail, manufacturing, services)
- Monthly revenue figure

Loan application endpoint

- Requested loan amount
- Repayment tenure in months
- Purpose of the loan

Decision engine endpoint

- Returns a binary decision: Approved or Rejected
- Returns a computed credit score (custom logic, clearly documented)
- Returns one or more reason codes (e.g. LOW_REVENUE, HIGH_LOAN_RATIO, DATA_INCONSISTENCY)

FE Frontend Interface

Required

- Single-page form collecting all required business profile and loan inputs
- Decision result view showing: approval status, credit score, and reason codes
- Clear and functional UI — production quality preferred but not over-engineered

DL Decision Logic

Required

You are free to design your own scoring model. Examples of signal you may use:

- Revenue-to-EMI ratio (monthly revenue vs estimated monthly repayment)
- Loan amount as a multiple of monthly revenue
- Tenure-adjusted risk (very short or very long tenures)
- Basic fraud and consistency checks (e.g. loan amount disproportionate to revenue)

Document all thresholds and assumptions in your README. The logic itself is less important than your ability to explain and defend it.

EC Edge Case Handling

Required

Your system must gracefully handle the following real-world input conditions:

- Missing or partially incomplete form fields
- Invalid input formats (e.g. negative revenue, non-numeric values, malformed PAN)
- Conflicting data (e.g. ₹10L monthly revenue with ₹5Cr loan request)

★ Bonus — Optional but Recommended

Optional

- Async decision processing — simulate a background job or queue with status polling or webhooks
- Audit trail — log all application submissions and decisions with timestamps
- API validation middleware and structured error responses
- Rate limiting on decision endpoints
- Docker Compose setup for local development

02 TECHNICAL EXPECTATIONS

AREA	WHAT WE LOOK FOR	WEIGHT
API design	RESTful conventions, meaningful route structure, consistent response schema	High
Code quality	Modular, readable, appropriately abstracted — no god functions or monolithic files	High
Decision logic	Correctness, documented assumptions, sensible thresholds, defensible reasoning	Medium
Error handling	Graceful degradation, informative error codes, no unhandled exceptions in production paths	Medium
Documentation	Clear README, setup instructions that work, API reference, assumptions listed	Standard

03 DELIVERABLES

01 GitHub Repository Public or shared-access repo with a clean commit history. Do not squash everything into one commit.	02 Deployed Preview Live URL via Vercel, Netlify, Render, or equivalent. Both frontend and backend must be accessible.	03 README Setup guide, API docs, decision logic explanation, edge case strategy, and assumptions made.	04 Write-up PDF 1–2 pages. Architecture decisions, tradeoffs chosen, and what you'd improve with more time.
---	---	---	--

This is a confidential assessment issued by Vitto. Do not distribute or publish publicly.

Questions? Reach out to your recruiting contact before starting.